Zen-Reasoning:

Verifiable Chain-of-Thought with Formal Guarantees

Antje Worring, Zach Kelling, **Hanzo AI Research**[1] **Zoo Labs Foundation**[2]

[1]Hanzo AI Inc. (Techstars '17) [2]Zoo Labs Foundation (501(c)(3))

research@hanzo.ai foundation@zoo.ngo

https://hanzo.ai/research/zen-reasoning

February 2026

Abstract

We present **Zen-Reasoning**, a language model specialized for verifiable mathematical and logical reasoning with formal guarantees on solution correctness. Unlike standard chain-of-thought (CoT) approaches where reasoning steps may contain subtle errors that propagate to incorrect conclusions, Zen-Reasoning integrates step-level verification through a novel **Verify-then-Generate (VtG)** paradigm that validates each reasoning step against formal proof assistants before proceeding. The system combines three key innovations: (1) a **Step-Level Verifier (SLV)** that decomposes informal reasoning into verifiable logical steps and checks each against Lean 4 and Z3 solvers, (2) a **Self-Correction Module (SCM)** that identifies and repairs verification failures through targeted backtracking and alternative reasoning paths, and (3) a **Tool-Augmented Reasoning Engine (TARE)** that dynamically invokes computational tools—symbolic algebra systems, numerical solvers, code interpreters, and web search—when pure language model reasoning is insufficient. Zen-Reasoning achieves state-of-the-art results on GSM8K (97.8% accuracy), MATH (86.4%), GPQA (61.8%), and the ARC-AGI public evaluation (44.2%), while providing formal verification certificates for 73.2% of its correct solutions on MATH. On a new benchmark of competition mathematics problems (AMC/AIME level), Zen-Reasoning solves 68.4% of problems with verified proofs, compared to 41.2% for the next best system. We demonstrate that formal verification not only provides correctness guarantees but also improves accuracy by 4–8% through early detection and correction of reasoning errors.

Keywords: Chain-of-Thought Reasoning, Formal Verification, Lean 4, Self-Correction, Tool-Augmented Reasoning, Mathematical Problem Solving

1 Introduction

Large language models have demonstrated remarkable reasoning capabilities through chain-of-thought (CoT) prompting (Wei et al., 2022), where models generate intermediate reasoning steps before arriving at a final answer. This approach has yielded significant improvements on mathematical reasoning (Lewkowycz et al., 2022), scientific question answering (Sun et al., 2024), and logical deduction (Saparov and He, 2023) tasks. However, CoT reasoning remains fundamentally unreliable: models can produce chains that appear plausible but contain subtle logical errors, incorrect calculations, or unjustified inferential leaps that propagate to wrong conclusions.

The unreliability of neural reasoning poses a critical barrier to deploying language models in high-stakes domains—mathematical research, scientific discovery, engineering design, legal analysis—where correctness is non-negotiable. A calculator that returns wrong answers 15% of the time is not merely imperfect; it is useless.

Zen-Reasoning addresses this fundamental limitation by integrating formal verification directly into the reasoning process. Rather than generating a complete chain of thought and checking it post hoc, Zen-Reasoning verifies each step as it is produced, catching errors before they can propagate. When verification fails, the self-correction module identifies the source of the error and generates alternative reasoning paths. When neural reasoning alone is insufficient, the tool-augmented engine brings in external computational resources.

The key insight is that formal verification and neural reasoning are complementary: language models excel at creative problem decomposition, intuitive leaps, and natural language understanding, while formal systems excel at precise logical deduction and exhaustive case analysis. Zen-Reasoning combines both within a unified architecture that can flexibly allocate reasoning effort between neural and formal modes.

Our contributions are:

1. The Verify-then-Generate (VtG) paradigm, which interleaves neural reasoning with formal verification at the step level.

2. A Step-Level Verifier that translates informal reasoning steps into Lean 4 proof obligations and checks them with automated tactics.

3. A Self-Correction Module that repairs verification failures through targeted backtracking with provably terminating search.

4. A Tool-Augmented Reasoning Engine with dynamic tool selection and result integration.

5. State-of-the-art results on GSM8K, MATH, GPQA, and ARC-AGI, with formal verification certificates for the majority of correct solutions.

2 Background and Related Work

2.1 Chain-of-Thought Reasoning

Chain-of-thought prompting (Wei et al., 2022) elicits step-by-step reasoning from language models by providing few-shot examples with intermediate steps. Zero-shot CoT (Kojima et al., 2022) achieves similar effects by appending "Let's think step by step" to prompts. Self-consistency (Wang et al., 2023) improves reliability by sampling multiple reasoning chains and taking majority vote. Tree-of-Thought (Yao et al., 2024) and Graph-of-Thought (Besta et al., 2024) extend CoT to non-linear reasoning structures.

Despite these advances, CoT reasoning remains prone to errors including arithmetic mistakes, logical fallacies, incorrect lemma application, and hallucinated intermediate results (Huang et al., 2024).

2.2 Formal Verification and Proof Assistants

Interactive theorem provers such as Lean 4 (de Moura et al., 2021), Coq (Bertot and Castéran, 2013), and Isabelle (Paulson, 1994) provide mechanically verified proofs of mathematical statements. These systems guarantee correctness by checking every deductive step against a small, trusted kernel. Recent work has explored using language models to generate formal proofs (Polu and Sutskever, 2020; Lample et al., 2022; Yang et al., 2024), achieving notable success on olympiad-level problems.

SMT solvers such as Z3 (de Moura and Bjørner, 2008) provide automated decision procedures for specific logical theories (linear arithmetic, bitvectors, arrays), complementing the interactive verification provided by proof assistants.

2.3 Self-Correction in Language Models

Self-correction mechanisms enable models to identify and fix errors in their own outputs. Self-Refine (Madaan et al., 2024) uses iterative feedback-generation loops. Reflexion (Shinn et al., 2024) maintains an episodic memory of past failures. However, research has shown that without external verification, language models often fail to reliably identify their own errors (Huang et al., 2024), sometimes introducing new errors while attempting to fix existing ones.

2.4 Tool-Augmented Language Models

Toolformer (Schick et al., 2024) trained models to use APIs through self-supervised learning. PAL (Gao et al., 2023) and PoT (Chen et al., 2023) delegate computation to code interpreters. Chameleon (Lu et al., 2024) composes multiple tools for complex reasoning. These approaches demonstrate that offloading computation to specialized tools can dramatically improve accuracy on tasks requiring precise calculation.

Algorithm 1 Verify-then-Generate Reasoning

Require: Problem P, max steps N, max corrections K

```
C ← []                                          ▷ Reasoning chain
S ← {P}                                         ▷ Formal state
for n = 1, ..., N do
    s_n ← Generate(P, C, S)                     ▷ Generate next step
    if s_n requires tool use then
        s_n ← TARE(s_n, C)                      ▷ Execute tool call
    end if
    (v, σ) ← SLV(s_n, S)                        ▷ Verify step
    if v = VALID then
        C ← C ∪ {s_n}
        S ← S ∪ {σ}                             ▷ Update formal state
    else
        for k = 1, ..., K do                    ▷ Self-correction
            s'_n ← SCM(s_n, v, C, S)
            (v', σ') ← SLV(s'_n, S)
            if v' = VALID then
                C ← C ∪ {s'_n}; S ← S ∪ {σ'}
                break
            end if
        end for
        if all corrections failed then
            Backtrack to last verified step
        end if
    end if
    if s_n is a final answer then
        return (C, S)                           ▷ Solution with proof
    end if
end for
```

3 Architecture

Zen-Reasoning integrates three modules—the Step-Level Verifier (SLV), Self-Correction Module (SCM), and Tool-Augmented Reasoning Engine (TARE)—with a Zen-72B backbone model fine-tuned for mathematical and logical reasoning. We describe each component and their interaction.

3.1 System Overview

Given a problem P, Zen-Reasoning generates a solution through the following iterative process:

3.2 Step-Level Verifier (SLV)

The SLV validates each reasoning step by translating it into a formal obligation and checking it against proof assistants.

Step Classification. Each reasoning step s_n is first classified into one of five categories:

- **Algebraic manipulation:** Equation transformations, simplifications, substitutions
- **Logical deduction:** Implications, case analysis, proof by contradiction
- **Numerical computation:** Arithmetic, evaluation of expressions
- **Definitional:** Introduction of variables, notation, or problem restatement
- **Heuristic:** Intuitive leaps, pattern recognition, strategy selection

The first three categories are formally verifiable; definitional steps are checked for consistency; heuristic steps are flagged as unverified but tracked for potential backtracking.

Autoformalization. The SLV translates informal reasoning steps into Lean 4 proof obligations using a specialized autoformalization model $\mathcal{A}_\phi$:

$$\tau_n = \mathcal{A}_\phi(s_n, \mathcal{S}_{\text{formal}}) \tag{1}$$

where τ_n is a Lean 4 tactic proof term and $\mathcal{S}_{\text{formal}}$ is the current formal context (accumulated hypotheses and definitions). The autoformalization model is a Zen-7B model fine-tuned on 2.8 million (informal step, Lean 4 tactic) pairs extracted from Mathlib4 and ProofNet.

Verification Strategies. Depending on the step category, the SLV applies different verification strategies:

1. **Lean 4 tactic proof:** For algebraic and logical steps, the autoformalized tactic is executed in the Lean 4 kernel. If the tactic succeeds, the step is verified. If it fails, the SLV attempts up to 5 alternative tactic suggestions from the autoformalization model.
2. **Z3 SMT solving:** For steps involving linear arithmetic, polynomial inequalities, or propositional logic, the obligation is encoded as an SMT formula and checked with Z3:
$$\text{Z3}(\neg(\mathcal{S}_n \implies s_n)) = \textsc{Unsat} \implies s_n \text{ is valid} \tag{2}$$
3. **Numerical verification:** For computational steps, the SLV evaluates the claimed computation using arbitrary-precision arithmetic (via mpmath) and checks agreement within a tolerance of 10^{-12}.
4. **Consistency checking:** For definitional steps, the SLV verifies that new definitions do not contradict existing hypotheses using a lightweight constraint solver.

Confidence Scoring. Each verified step receives a confidence score:

$$c_n = \begin{cases} 1.0 & \text{if formally verified (Lean 4 or Z3)} \\ 0.95 & \text{if numerically verified} \\ 0.8 & \text{if consistency-checked} \\ p_{\text{model}}(s_n|\mathcal{C}, P) & \text{if heuristic (model confidence)} \end{cases} \tag{3}$$

The overall solution confidence is $C = \prod_{n=1}^{N} c_n$.

3.3 Self-Correction Module (SCM)

When verification fails, the SCM identifies the error and generates corrections.

Error Diagnosis. The SCM receives the failed step s_n, the verification result v (including the specific failure mode from Lean 4 or Z3), and the current reasoning context. It generates a structured error diagnosis:

$$\text{diag} = \text{SCM}_{\text{diagnose}}(s_n, v, \mathcal{C}, \mathcal{S}) \tag{4}$$

Common failure modes include:

- **Arithmetic error:** Incorrect calculation (e.g., $3 \times 7 = 24$)
- **Sign error:** Incorrect handling of negative values
- **Missing case:** Incomplete case analysis (e.g., forgetting the negative root)
- **Invalid lemma:** Application of a theorem whose preconditions are not met
- **Logical gap:** A step that does not follow from the premises

Targeted Repair. Based on the diagnosis, the SCM generates a corrected step by conditioning on both the error analysis and the formal feedback:

$$s'_n = \text{SCM}_{\text{repair}}(s_n, \text{diag}, v_{\text{formal}}, \mathcal{C}) \tag{5}$$

where v_{formal} includes the specific Lean 4 error message or Z3 counterexample, providing precise information about why the step failed.

Backtracking. If correction fails after K attempts, the SCM initiates backtracking to the last verified step and generates an alternative reasoning path. To prevent infinite loops, we maintain a set of explored paths and use a beam search with diversity penalty:

$$\text{score}(\mathcal{C}') = \log p(\mathcal{C}'|P) - \lambda \sum_{\mathcal{C}_j \in \text{explored}} \text{sim}(\mathcal{C}', \mathcal{C}_j) \tag{6}$$

where $\lambda = 0.3$ penalizes similarity to previously explored chains.

Proposition 3.1 (Termination)**.** *The VtG reasoning process terminates in at most $N \times (K+1) \times B$ verification calls, where N is the maximum chain length, K is the maximum corrections per step, and B is the maximum backtracking depth.*

3.4 Tool-Augmented Reasoning Engine (TARE)

TARE extends the model's reasoning capabilities by dynamically invoking external tools.

Available Tools.

- **SymPy:** Symbolic algebra, calculus, equation solving
- **SageMath:** Number theory, combinatorics, abstract algebra
- **Python interpreter:** General computation, simulation, brute-force search
- **Wolfram Alpha API:** Mathematical knowledge base queries
- **Lean 4 REPL:** Interactive theorem proving for exploration
- **Web search:** Retrieval of mathematical facts and theorems

Tool Selection. The model learns to select appropriate tools through a routing mechanism:

$$t^* = \arg\max_{t \in \mathcal{T}} p_\theta(t | s_n, \mathcal{C}, P) \tag{7}$$

where $\mathcal{T}$ is the set of available tools and p_θ is the tool selection probability predicted by the backbone model. Tool selection is trained through supervised learning on 500K examples of correct tool usage.

Result Integration. Tool outputs are formatted as verified facts and integrated into the reasoning chain:

$$s_n^{\text{tool}} = \text{Format}(t^*, \text{output}(t^*, \text{query}(s_n))) \tag{8}$$

Tool-computed results receive a confidence of 1.0 (for symbolic algebra) or 0.98 (for numerical computation), as they are produced by reliable external systems.

4 Training

4.1 Data Curation

Training data for Zen-Reasoning is curated from four sources:

Table 1: Training data composition for Zen-Reasoning.

Source	Description	Examples	Verified
Mathlib4	Lean 4 library theorems	420K	100%
ProofNet	Formal-informal proof pairs	180K	100%
AMPS	Synthetic math problems	5.2M	92%
Competition math	AMC, AIME, IMO, Putnam	48K	85%
GSM-hard	Augmented grade school math	1.2M	97%
STEM textbooks	University-level problems	680K	78%
Tool-use traces	Correct tool invocations	500K	100%
Error-correction	(error, diagnosis, fix) triples	340K	100%
Total		**8.57M**	

Verification Pipeline. Training examples undergo automated verification: solutions are parsed into individual steps, each step is autoformalized and checked against Lean 4/Z3, and only examples where all verifiable steps pass are retained in the "verified" category. Examples with verification failures are used to generate error-correction training data.

Synthetic Data Generation. We generate 5.2M synthetic problems using a curriculum of increasing difficulty, from single-step arithmetic to multi-step algebraic proofs. Each synthetic problem is generated with both a step-by-step solution and a formal proof, ensuring ground-truth verification.

4.2 Training Procedure

Zen-Reasoning is initialized from the aligned Zen-72B checkpoint and fine-tuned in three phases:

Phase 1: Reasoning Pre-training (2 weeks, 128 A100 GPUs). The backbone is fine-tuned on the full 8.57M example dataset using the SFT objective. We train for 3 epochs with learning rate 5×10^{-6}, batch size 128, and cosine schedule.

Phase 2: Verification-Aware Training (1 week, 128 A100 GPUs). We introduce the VtG loop during training, using online verification to provide step-level feedback. The model receives additional reward for steps that pass formal verification:

$$r_n = \begin{cases} +1.0 & \text{step verified and correct} \\ -0.5 & \text{step verified but unnecessary} \\ -1.0 & \text{step failed verification} \\ +0.5 & \text{heuristic step leading to verified conclusion} \end{cases} \tag{9}$$

We use this reward signal with PPO to optimize the policy for generating verifiable reasoning chains.

Phase 3: Self-Correction Training (3 days, 64 A100 GPUs). The SCM is trained on the error-correction dataset (340K examples). For each example, the model sees a failed step, the verification error message, the correct diagnosis, and the corrected step. We fine-tune with the SFT objective on this structured data.

4.3 Autoformalization Training

The autoformalization model $\mathcal{A}_\phi$ is trained separately on 2.8M (informal, formal) pairs:

- 420K examples from Mathlib4 (docstring to tactic)
- 180K examples from ProofNet (natural language to Lean 4)
- 1.2M synthetic examples generated by back-translation (Lean 4 $\rightarrow$ informal $\rightarrow$ Lean 4)
- 1.0M examples from augmented textbook proofs

The autoformalization model achieves 78.4% exact-match accuracy on a held-out test set of 10K (informal, formal) pairs, and 91.2% accuracy when allowing up to 5 tactic suggestions per step.

5 Evaluation

We evaluate Zen-Reasoning on four established benchmarks and two new benchmarks, measuring both accuracy and verifiability.

5.1 Benchmarks

- **GSM8K** (Cobbe et al., 2021): 1319 grade-school math word problems requiring multi-step arithmetic reasoning.
- **MATH** (Hendrycks et al., 2021): 5000 competition-level math problems across 7 categories (Prealgebra, Algebra, Number Theory, Counting & Probability, Geometry, Intermediate Algebra, Precalculus) with 5 difficulty levels.
- **GPQA** (Rein et al., 2024): 448 graduate-level science questions validated by domain experts.
- **ARC-AGI** (Chollet, 2019): Abstract reasoning tasks requiring pattern recognition and rule induction.
- **CompMath (new):** 500 competition mathematics problems at AMC 10/12 and AIME difficulty, with human-verified solutions and formal proofs.
- **ProofBench (new):** 200 theorem-proving tasks requiring complete Lean 4 proofs, spanning undergraduate to graduate mathematics.

Table 2: Main results on mathematical and scientific reasoning benchmarks.

Method	GSM8K	MATH	GPQA	ARC-AGI	CompMath
Zen-72B (CoT)	95.2	78.6	52.3	38.2	48.6
GPT-4o (CoT)	94.8	76.3	53.1	36.8	45.2
o1-preview	96.4	83.2	58.4	42.1	62.8
DeepSeek-R1	97.1	84.8	59.2	41.8	61.4
Zen-Reasoning	**97.8**	**86.4**	**61.8**	**44.2**	**68.4**
w/o SLV	96.1	81.2	55.8	40.1	56.2
w/o SCM	96.8	83.7	58.4	42.3	62.1
w/o TARE	97.2	84.1	57.2	43.8	64.8

5.2 Metrics

- **Accuracy:** Fraction of problems solved correctly (final answer matches ground truth).
- **Verified Accuracy:** Fraction of correct solutions accompanied by a complete formal proof.
- **Verification Rate:** Fraction of reasoning steps that are formally verified.
- **Self-Correction Rate:** Fraction of initially incorrect steps successfully repaired by the SCM.
- **Tool Usage Rate:** Fraction of problems where at least one external tool is invoked.

5.3 Baselines

- **Zen-72B (CoT):** The base Zen-72B model with standard chain-of-thought prompting.
- **GPT-4o (CoT):** OpenAI's GPT-4o with chain-of-thought prompting.
- **o1-preview:** OpenAI's reasoning model with extended thinking.
- **DeepSeek-R1:** DeepSeek's reasoning model.
- **AlphaProof:** DeepMind's formal reasoning system (where results are available).

5.4 Main Results

Table 2 shows Zen-Reasoning achieves state-of-the-art on all five benchmarks. The most significant gains are on MATH (+1.6% over DeepSeek-R1) and CompMath (+5.6% over o1-preview), where formal verification catches errors that escape informal reasoning.

Ablation results confirm that each component contributes meaningfully: removing the SLV causes the largest accuracy drop (-5.2% on MATH), demonstrating that step-level verification is the primary driver of improvement. The SCM contributes -2.7% and TARE contributes -2.3% on MATH.

Table 3: Verification statistics on MATH benchmark by category.

Category	Accuracy	Verified	Verif. Rate	SC Rate
Prealgebra	98.2	92.1	94.3%	23.1%
Algebra	93.4	84.2	88.7%	31.4%
Number Theory	84.2	71.8	78.4%	28.7%
Count. & Prob.	82.1	65.3	72.1%	35.2%
Geometry	78.6	58.4	64.2%	41.3%
Inter. Algebra	81.3	68.7	74.8%	33.8%
Precalculus	79.8	63.2	68.3%	37.6%
Overall	**86.4**	**73.2**	**77.3%**	**32.4%**

Table 4: MATH accuracy by difficulty level (1 = easiest, 5 = hardest).

Method	Level 1	Level 2	Level 3	Level 4	Level 5
Zen-72B (CoT)	97.1	92.4	83.2	71.6	52.8
o1-preview	98.4	94.1	87.8	78.3	61.4
DeepSeek-R1	98.8	95.2	89.1	79.8	63.2
Zen-Reasoning	**99.2**	**96.8**	**91.4**	**83.2**	**68.4**

5.5 Verification Analysis

Table 3 breaks down verification statistics by MATH category. Algebraically-heavy categories (Prealgebra, Algebra) achieve the highest verification rates, while geometry problems—which often require spatial reasoning that is harder to formalize—have lower verification rates. The self-correction rate of 32.4% indicates that roughly one-third of initially incorrect steps are successfully repaired.

5.6 MATH Results by Difficulty

The advantage of formal verification increases with problem difficulty: on Level 5 (hardest) problems, Zen-Reasoning outperforms DeepSeek-R1 by 5.2%, compared to only 0.4% on Level 1 problems. This confirms that verification is most valuable for complex reasoning chains where errors are most likely to accumulate.

5.7 ProofBench Results

On ProofBench, Zen-Reasoning solves 58.5% of formal proof tasks, outperforming AlphaProof (52.0%, where comparable) while being 3.6× faster. The combination of neural proof search with formal verification produces more efficient proofs than purely search-based approaches.

5.8 Error Analysis

We analyze the 680 incorrectly solved MATH problems:

The largest error category (32.1%) is formalization failure—problems where the

Table 5: Formal proof generation on ProofBench (200 problems).

Method	Solved	Avg Steps	Avg Time (s)	Tactics/Step
Lean Copilot	31.0%	8.2	42.3	1.4
LeanDojo	38.5%	12.1	68.7	1.8
AlphaProof*	52.0%	6.4	124.8	2.1
Zen-Reasoning	**58.5%**	9.8	34.2	1.6

Table 6: Error analysis on MATH (680 incorrect solutions).

Error Type	Count	% of Errors
Formalization failure (cannot verify)	218	32.1%
Incorrect heuristic step (unverifiable)	156	22.9%
Problem misunderstanding	112	16.5%
Exhausted correction budget	89	13.1%
Tool error / incorrect tool choice	54	7.9%
Correct solution, wrong final answer format	51	7.5%

autoformalization model cannot translate informal reasoning into Lean 4 tactics. This represents the primary bottleneck and the most promising avenue for improvement.

6 Ablation Studies

6.1 Verification Backend

The cascade strategy—trying Z3 first (fast, covers arithmetic and propositional logic) then falling back to Lean 4 (slower, covers higher-order reasoning)—provides the best accuracy-overhead trade-off.

6.2 Correction Budget

Accuracy improves with correction budget up to $K = 5$, beyond which diminishing returns set in. Most correctable errors are fixed within 3 attempts.

6.3 Tool Usage Analysis

SymPy is the most frequently used and most impactful tool, providing symbolic computation that complements neural reasoning. The Lean 4 REPL is used for exploratory proof search, helping the model discover proof strategies.

7 Discussion

7.1 Why Verification Improves Accuracy

Formal verification improves accuracy through three mechanisms:

Table 7: Effect of verification backend on MATH accuracy.

Backend	Accuracy	Verif. Rate	Overhead (s)
No verification	78.6	0%	0
Z3 only	82.1	41.2%	1.2
Lean 4 only	84.8	68.4%	8.4
Z3 + Lean 4 (cascade)	**86.4**	**77.3%**	5.8

Table 8: Effect of self-correction budget K on MATH accuracy.

K (corrections)	Accuracy	Avg Time (s)	SC Rate
0 (no correction)	82.8	12.4	0%
1	84.6	18.2	22.1%
3	86.1	24.8	30.8%
5 (default)	**86.4**	28.3	32.4%
10	86.5	41.7	33.1%

1. **Error prevention:** Step-level verification catches errors immediately, preventing them from propagating through the reasoning chain. On MATH, 18.3% of initially generated steps fail verification but are successfully corrected.
2. **Confidence calibration:** The model learns to produce more careful, verifiable reasoning steps during verification-aware training, reducing the initial error rate.
3. **Search guidance:** Verification failures provide structured feedback that guides the model toward correct reasoning paths more efficiently than unconstrained search.

7.2 Limitations

- **Autoformalization bottleneck:** The autoformalization model fails to translate 32.1% of incorrect solutions, representing the primary accuracy ceiling.
- **Geometry and spatial reasoning:** Geometric problems are inherently harder to formalize, leading to lower verification rates and accuracy.
- **Latency:** The VtG loop adds 2–5× latency compared to standard CoT generation, making it unsuitable for real-time applications.
- **Lean 4 dependency:** The system requires a running Lean 4 server, adding infrastructure complexity.
- **Domain specificity:** The current system is specialized for mathematical and logical reasoning; extending to other domains (legal, scientific) requires domain-specific formalization.

7.3 Broader Impact

Verifiable reasoning has significant implications for AI safety. By providing formal certificates of correctness, Zen-Reasoning enables *trustworthy AI reasoning*—users can

Table 9: Tool usage statistics on MATH by tool type.

Tool	Usage Rate	Success Rate	Acc. Improvement
SymPy	34.2%	96.1%	+3.8%
Python interpreter	22.8%	91.4%	+2.1%
SageMath	8.4%	93.2%	+1.2%
Lean 4 REPL	12.1%	82.3%	+1.8%
Web search	5.2%	78.6%	+0.4%

verify that the model's conclusions follow from its premises, rather than trusting the model on faith. This is particularly valuable in education (ensuring correct mathematical instruction), engineering (verifying design calculations), and scientific research (validating proof attempts).

8 Conclusion

We presented Zen-Reasoning, a language model that integrates formal verification into the reasoning process through the Verify-then-Generate paradigm. By validating each reasoning step against Lean 4 and Z3 solvers, detecting and correcting errors through self-correction, and augmenting reasoning with external tools, Zen-Reasoning achieves state-of-the-art results on GSM8K (97.8%), MATH (86.4%), GPQA (61.8%), and ARC-AGI (44.2%) while providing formal verification certificates for 73.2% of correct solutions.

The key lesson of this work is that formal verification is not merely a post-hoc auditing tool but an active component of the reasoning process that improves accuracy by 4–8% through early error detection and guided correction. We believe this approach points toward a future where AI reasoning is not just impressive but provably correct.

Models, verification infrastructure, and benchmarks are available at https://github.com/hanzoai/zen-reasoning under Apache 2.0.

References

Bertot, Y. and Castéran, P. *Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions*. Springer, 2013.

Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Gianinazzi, L., Gajber, J., Lehmann, T., Podstawski, M., Nyczyk, H., Wettig, A., and Hoefler, T. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI Conference on Artificial Intelligence*, 2024.

Chen, W., Ma, X., Wang, X., and Cohen, W. W. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *TMLR*, 2023.

Chollet, F. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., and Neubig, G. PAL: Program-aided language models. In *ICML*, pp. 10764–10799, 2023.

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS*, 2021.

Huang, J., Gu, S. S., Le Hou, Wu, Y., Wang, X., Yu, H., and Han, J. Large language models cannot self-correct reasoning yet. In *ICLR*, 2024.

Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. Large language models are zero-shot reasoners. In *NeurIPS*, 2022.

Lample, G., Lacroix, T., Lachaux, M.-A., Rodriguez, A., Hayat, A., Lavril, T., Ebner, G., and Martinet, X. HyperTree proof search for neural theorem proving. In *NeurIPS*, 2022.

Lewkowycz, A., Andreassen, A., Dohan, D., Dyer, E., Michalewski, H., Ramasesh, V., Slone, A., Anil, C., Schlag, I., Gutman-Solo, T., et al. Solving quantitative reasoning problems with language models. In *NeurIPS*, 2022.

Lu, P., Peng, B., Cheng, H., Galley, M., Chang, K.-W., Wu, Y. N., Zhu, S.-C., and Gao, J. Chameleon: Plug-and-play compositional reasoning with large language models. In *NeurIPS*, 2024.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., et al. Self-refine: Iterative refinement with self-feedback. In *NeurIPS*, 2024.

de Moura, L. and Bjørner, N. Z3: An efficient SMT solver. In *TACAS*, pp. 337–340, 2008.

de Moura, L., Kong, S., Avigad, J., van Doorn, F., and von Raumer, J. The Lean 4 theorem prover and programming language. In *CADE*, pp. 625–635, 2021.

Paulson, L. C. *Isabelle: A Generic Theorem Prover*. Springer, 1994.

Polu, S. and Sutskever, I. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.

Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. GPQA: A graduate-level Google-proof Q&A benchmark. In *ICLR*, 2024.

Saparov, A. and He, H. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. In *ICLR*, 2023.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2024.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2024.

Sun, R., Ren, H., and Liang, P. SciEval: A multi-level large language model evaluation benchmark for scientific research. In *AAAI*, 2024.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

Yang, K., Swope, A., Gu, A., Chalamala, R., Song, P., Yu, S., Godil, S., Prenger, R., and Anandkumar, A. LeanDojo: Theorem proving with retrieval-augmented language models. In *NeurIPS*, 2024.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2024.

A Autoformalization Examples

```
Informal: "Since x^2 - 5x + 6 = 0, we can factor
          to get (x-2)(x-3) = 0"

Lean 4:
  have h : x^2 - 5*x + 6 = (x - 2) * (x - 3) := by ring
  rw [h] at h_eq
```

Listing 1: Example: informal step to Lean 4 tactic

```
Informal: "The sum 1/2 + 1/3 + 1/6 = 1"

Verification:
  Compute: 1/2 + 1/3 + 1/6 = 3/6 + 2/6 + 1/6 = 6/6 = 1
  Status: VERIFIED (exact arithmetic)
```

Listing 2: Example: numerical verification

B CompMath Benchmark Details

CompMath contains 500 problems: 200 at AMC 10/12 level (answer is an integer 000–999), 200 at AIME level (answer is an integer 000–999), and 100 at USA(J)MO level (proof required). Problems were collected from public competition archives and verified by three independent mathematicians. Formal Lean 4 proofs are provided for 380 of the 500 problems.

C Computational Cost

Table 10: Average computational cost per problem by benchmark.

Benchmark	Steps	Verif. Calls	Tool Calls	Time (s)	Tokens
GSM8K	6.2	4.8	1.2	8.4	1,240
MATH	12.8	9.4	2.8	28.3	3,820
GPQA	8.4	5.2	3.1	22.1	4,560
ARC-AGI	14.2	8.1	5.4	34.8	2,180
CompMath	18.6	14.2	4.2	52.4	6,240

The verification overhead averages 2–5× compared to standard CoT generation, but this is offset by the accuracy improvement and the value of formal correctness certificates.